

Starman CRM Prototype Audit

Audit date: July 10, 2026

Artifact reviewed: Starman 5.0 package supplied to Aspire

Review method: Source-code and documentation review of the July 10 package. The applications were not run as part of this audit.

Audit finding

Starman 5.0 was a substantial single-user CRM prototype with a broad view of Aspire's advisory work. It brought client records, households, pipeline activity, tasks, planning calculations, meeting notes, compliance controls, and AI-assisted drafting into one interface. The prototype showed strong product thinking, especially in its use of human approval, evidence, access roles, and audit records around AI-assisted work.

The version available on July 10 was not ready for firm-wide use or real client data. Its working interface was one self-contained HTML file. Records were stored in the browser on one computer, there were no staff accounts or shared database, and clearing browser storage could remove the data. Its local AI features depended on Ollama running on Jack's MacBook.

A separate production foundation showed the intended route beyond the prototype: Node and Express, PostgreSQL through Prisma, role-based access, organization-scoped records, API keys, audit logging, and connector scaffolds. That foundation had authentication, leads, and intakes connected end to end. Most of the Starman 5.0 interface remained to be moved from browser storage to the production database.

The audit therefore found two related products:

- 1 A feature-rich design prototype that could demonstrate workflows and support discussion with staff.
- 2 An early production foundation that proved a shared, database-backed version was technically plausible.

The product had reached a credible prototype stage. Production use still required scope control, staff validation, technical implementation, security work, compliance approval, deployment, data migration, and a long-term maintenance owner.

Prototype journey and MVP evolution

The retained files show repeated expansion and refinement rather than a single build. They do not include a formal interview log, test record, or change log that ties each version to named staff feedback. The sequence below records what can be established from the source files and design documents.

Stage	Product state	Product decision visible in the artifacts
Version 1.01	Baseline Aspire CRM with a dashboard, clients, pipeline, tasks, reports, compliance, and sample household records.	Start with a single-file prototype so the main workflows and navigation could be built and reviewed quickly.
Versions 1.1 to 1.2	Added households, a compliance calendar, suitability flags, meeting preparation, document and note handling, insurance information, a more detailed Client 360, an advisor command centre, and a local compliance rules engine.	Expand from contact storage toward the daily work of an advisory firm. Use synthetic data and local rules to make workflows visible before building infrastructure.

Stage	Product state	Product decision visible in the artifacts
Version 4.0, Constellation rebuild	Reworked the interface around clients, households, book health, and relationship views. Added persistent browser storage, JSON import and export, responsive navigation, and a more deliberate Aspire-branded interface.	Treat household relationships and an advisor's whole book as central product concepts. Improve usability and visual consistency before production porting.
Version 4.5	Added suggest-only household matching, evidence review, approve or reject decisions, sensitive-data handling, and an audit trail.	Keep AI away from direct record changes. Require an advisor to review evidence and approve any relationship link.
Version 4.9	Added Meeting Intelligence and expanded the Advisor Agent. Transcripts could be ingested, matched to a client or household, reviewed, and filed. AI responses were organized as drafts and proposed actions.	Focus AI on preparation, extraction, summarization, and drafting. Keep consequential actions under human control and record decisions.
Version 5.0, Advisor OS	Combined the CRM, planning, compliance, meeting, reporting, calculator, and AI concepts into one broad advisor platform.	Use the prototype as a comprehensive product specification and demonstration environment.
Production foundation, Milestone 1	Added a server application, PostgreSQL data model, authentication, roles, organization scoping, audit records, API keys, and connector scaffolds. Leads and intakes were connected through the database.	Prove the architecture needed for shared use, then port prototype modules one at a time using a repeatable API pattern.

The evolution shows a move through three kinds of prototype:

- The first versions tested the scope and layout of a CRM for Aspire.
- The 4.x and 5.0 versions tested a larger advisor-workstation concept, including AI-assisted workflows.
- The production foundation tested whether the concept could become a shared application with controlled access and persistent records.

The package used a useful separation between exploration and production. The HTML prototype allowed rapid iteration. The server application established a safer technical direction without requiring every prototype feature to be rebuilt at once.

MVP definition as found

The term MVP applied differently to the two parts of the package.

The July 10 working MVP was the single-file Starman 5.0 prototype. It was sufficient to demonstrate the product concept, show representative workflows, expose assumptions, and collect staff reactions. Its sample data and local operation reduced the risk of using real client information during early exploration. It was also easy to open and review because it had no installation or build process.

The package's proposed production MVP was narrower than the full prototype. Its go-live plan prioritized:

- staff authentication and advisor or administrative roles;
- database-backed clients, households, pipeline, tasks, and compliance flags;
- a document repository and text-based Meeting Intelligence;
- an Advisor Agent limited to summaries, briefings, drafts, and proposed tasks;
- complete audit logging and CSV export;

- CSV import for an initial book of business;
- Canadian-hosted infrastructure and controlled AI processing.

Audio transcription, automatic household matching at scale, direct carrier connections, advanced analytics, multi-organization software, mobile access, and formal service certifications were placed after the initial pilot.

That scope choice was sensible. The full Starman 5.0 interface was too broad to use as the first production release. A pilot needed to prove that staff could sign in, find reliable records, complete a small set of frequent tasks, and trust the audit and access controls. AI features could then be added only where they saved time and retained human review.

The Milestone 1 backend had not yet reached that production MVP. It had the architectural pattern and a small working slice. Roughly two of the prototype's twelve main modules were connected through the production data layer. The rest remained browser-based designs.

Decisions, feedback, and testing record

Several product decisions were clear in the package:

- Human approval for AI work: AI output was treated as a suggestion, draft, extraction, or proposed action. The designs prevented the model from silently changing client, KYC, beneficiary, account, or suitability records.
- Evidence before approval: Household suggestions were designed to show the source evidence, counter-evidence, confidence factors, and sensitive-data flags. Beneficiary-derived links required client confirmation.
- Auditability: The prototype and production design recorded model calls, suggestions, views, decisions, logins, data changes, and connector actions. The audit model included the actor, role, time, and action.
- Role separation: Owners, advisors, assistants, compliance staff, and system administrators had different permissions. Administrative access was designed around system configuration rather than unrestricted access to client content.
- Production data outside the browser: The server foundation replaced browser storage with organization-scoped PostgreSQL records and API calls. This was the main architectural decision required for team use.
- Restricted integrations: Canada Life and Quadrus connectors were disabled in code pending official access and compliance approval. The available connector work used simulated calls, CSV, webhooks, or public integration patterns.
- Provider-flexible AI: The backend could remain in simulated mode or connect to a local or cloud model through a server-side provider layer. Model credentials were intended to stay off staff browsers.
- Canadian deployment target: Planning documents proposed Canadian cloud regions for the database, files, compute, backups, encryption keys, and AI processing. Specific vendor and model claims still required current verification before purchase or deployment.

The retained package did not show which features came from direct Aspire staff requests, which came from observation, or which came from AI-assisted ideation. It also did not contain recorded usability sessions, acceptance criteria, defect logs, performance results, security tests, or a completed compliance review. Source comments described the prototype as tested, and the presence of multiple versions shows active iteration. The audit could not verify the depth or results of that testing.

Future iterations should keep a short product record for each release: the problem observed, the staff member or workflow that supplied the evidence, the change made, the acceptance check, the result, and any follow-up. That record would make product decisions traceable and reduce the risk of carrying prototype assumptions into production.

Usefulness to Aspire

Starman was already useful as a structured model of Aspire's work. It gathered related activities that otherwise span client records, household relationships, task lists, calendars, forms, calculations, meeting notes, and compliance review. The prototype could help staff identify which information belongs together and which steps require clearer ownership.

The strongest near-term uses were:

- reviewing the proposed client, household, pipeline, task, and compliance workflows with the people who perform them;
- comparing the prototype with Aspire's current systems and identifying duplicate entry;
- deciding which three to five workflows belong in a first shared pilot;
- testing whether meeting preparation, follow-up drafting, and task creation produce enough time savings to justify the added system;
- inventorying the built-in calculators before creating overlapping tools;
- using the prototype as a visual reference while the production modules are rebuilt.

The built-in calculator set included RRIF minimum withdrawal, investment growth, RESP with CESG, GIC, and Canadian mortgage calculations. The Financial Planning area also included retirement projections, a sustainable-income solver, sequence-of-returns scenarios, RRIF schedules, and a 2025 Alberta and federal income-tax model. Each calculation would need a named source, effective date, test cases, and an upkeep owner before staff relied on it with clients.

Long-term value depends on whether Aspire wants an internal operating tool or a replacement for a commercial CRM. An internal tool can focus tightly on Aspire's workflows and may reduce repetitive administration. A full replacement carries a larger obligation: secure hosting, backups, recovery, access management, monitoring, privacy documentation, regulatory review, vendor and carrier constraints, data migration, calculator maintenance, user support, and continuing software updates.

Andrew's planning comparison was approximately \$1,250 per person per year for Maximizer, or about \$10,000 per year for eight users. That figure was staff-reported and had not been verified against a current quote, edition, or final seat count. Starman's own plan estimated approximately \$150 to \$400 per month for infrastructure, plus AI usage and development labour. A build-versus-buy decision should compare total annual cost, implementation risk, functionality, data portability, support, and the value of custom workflows.

Production gaps and recommended sequence

The July 10 package had the following production gaps:

- most interface modules were still connected only to browser storage;
- no shared staff deployment or production identity system existed;
- local AI depended on one developer's MacBook;
- the prototype's data could be lost through browser storage clearing or device failure;
- infrastructure files included development credentials and were not production configuration;
- backups, restore tests, monitoring, incident response, retention, and support procedures were not operating;
- real-data import had not been designed, validated, approved, or rehearsed;
- privacy, dealer, compliance, security, and integration approvals were incomplete;

- the package lacked documented staff acceptance testing and release evidence;
- ongoing ownership and maintenance effort were not yet formalized.

The recommended sequence is:

- 1 Name the product owner, technical maintainer, compliance approver, and final decision-maker.
- 2 Select three to five frequent workflows for the first pilot and define measurable acceptance checks with the staff who perform them.
- 3 Confirm which existing system remains the authoritative client record during the pilot.
- 4 Port only the selected workflows to the database-backed application.
- 5 Establish Canadian hosting, staff accounts, least-privilege roles, encryption, backups, restore testing, monitoring, and secret management.
- 6 Complete privacy, security, dealer, and compliance review before any real client data enters the system.
- 7 Test with synthetic data, then a controlled and approved data set, with defects and staff feedback recorded by release.
- 8 Train a small pilot group, measure use and time saved, and decide whether to expand, revise, or stop.

Ownership and maintenance outlook

The July 10 audit identified continuity as a major product risk because the prototype and local AI environment depended heavily on Jack. A current staffing path may reduce that risk: Andrew has discussed keeping Jack part-time in the fall. The arrangement was under discussion and had not been finalized when this note was prepared.

Part-time continuity would give the CRM a likely owner who understands its history and architecture. Aspire would still need to make the role explicit. The maintenance record should name who can deploy changes, respond to incidents, restore backups, manage access, update calculators and tax values, review dependencies, maintain cloud accounts, coordinate compliance approval, and support staff.

The tool should also be supportable without relying on one person's computer or memory. Source control, setup instructions, environment records, release notes, automated tests, deployment scripts, backup procedures, and a recovery exercise are part of the product. A second person should be able to follow the documentation and restore a working system.

As found on July 10, Starman was a strong product prototype and an early production build. Its best next step was a narrow, staff-tested pilot built on the production foundation, with formal ownership and approval established before real client information was introduced.